Acctos AI — Technical Documentation

Version: 1.0 **Date:** 16 March 2026 **Classification:** Internal / Engineering

---

Table of Contents

1. System Overview
2. Architecture
3. Technology Stack
4. Database Schema
5. Authentication & Authorization
6. API Reference
7. Core Business Logic
8. Third-Party Integrations
9. Frontend Application
10. Deployment & Configuration
11. Error Handling & Validation
12. Security Considerations

---

1. System Overview

Acctos AI is a multi-tenant SaaS platform that serves as the client-facing dashboard for AI Assist BG's document processing services. It enables organisations to monitor their document processing usage (PDF pages and Excel rows), manage billing and subscriptions, configure integrations with Make.com and Azure Document Intelligence, and submit support tickets.

Key Capabilities

- **Real-time usage monitoring** — track PDF pages processed and Excel rows extracted across billing periods
- **Multi-tenant isolation** — complete data separation per organisation with role-based access control

- **Automated workflow management** — auto-pause/resume Make.com scenarios when usage limits are exceeded or restored
- **Subscription & add-on billing** — Stripe-integrated billing with tiered subscription plans and one-time add-on purchases
- **Support ticketing** — in-app ticket creation with threaded messages, internal notes, and attachment support
- **Audit logging** — comprehensive action tracking for compliance and debugging

---

2. Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Client Browser                              │
│                     React 18 SPA (Vite)                              │
│              Pages: Dashboard | Billing | Tickets | Users            │
└──────────────────────────┬──────────────────────────────────────────┘
                           │ HTTPS (JWT Bearer)
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│                       Express.js API Server                          │
│                                                                      │
│  Middleware:  authenticateToken → requireRole → route handlers       │
│  Routes:     /api/auth  /api/usage  /v1/billing  /v1/events         │
│              /v1/integrations  /v1/tickets  /v1/users  /v1/usage     │
│                                                                      │
│  Utils:      usageLimits.ts (pause/resume/reset/limit checks)        │
└──────────┬──────────────┬──────────────┬──────────────────────────┘
           │              │              │
           ▼              ▼              ▼
┌──────────────┐ ┌────────────┐ ┌──────────────────┐
│  PostgreSQL  │ │  Stripe    │ │  Make.com API    │
│  (Prisma)    │ │  Webhooks  │ │  (eu2.make.com)  │
└──────────────┘ └────────────┘ └──────────────────┘
                                          │
                                          ▼
                                 ┌──────────────────┐
                                 │ Azure Document   │
                                 │ Intelligence     │
                                 └──────────────────┘
```

2.2 Project Structure (Monorepo)

```
acctos-ai/
├── apps/
│   ├── api/                      # Express backend
│   │   ├── prisma/
│   │   │   ├── schema.prisma     # Database schema
│   │   │   └── migrations/       # Prisma migration files
│   │   └── src/
│   │       ├── index.ts          # Entry point, server bootstrap
│   │       ├── routes/
│   │       │   ├── auth.ts       # Authentication endpoints
│   │       │   ├── billing.ts    # Billing, usage-status, Stripe webhook
│   │       │   ├── events.ts     # HMAC-secured event ingestion
│   │       │   ├── integrations.ts # Make.com and Azure integration
│   │       │   ├── tickets.ts    # Support ticket CRUD
│   │       │   ├── usage.ts      # Usage summary, timeseries, exports
│   │       │   └── users.ts      # User management
│   │       ├── middleware/
│   │       │   ├── auth.ts       # JWT verification, role enforcement
│   │       │   ├── hmac.ts       # HMAC signature verification
│   │       │   └── errorHandler.ts # Centralised error processing
│   │       ├── utils/
│   │       │   ├── usageLimits.ts  # Billing period, pause/resume, limit checks
│   │       │   └── roles.ts      # Role hierarchy definitions
│   │       └── types/
│   │           └── index.ts      # TypeScript interfaces
│   └── web/                      # React frontend
│       └── src/
│           ├── App.tsx           # Router, context providers
│           ├── pages/
│           │   ├── LandingPage.tsx  # Login / registration
│           │   ├── Dashboard.tsx    # Usage monitoring
│           │   ├── Billing.tsx      # Subscriptions, add-ons, limits
│           │   ├── Tickets.tsx      # Support tickets
│           │   └── Users.tsx        # User management
│           ├── components/
│           │   └── Layout.tsx       # Navigation, scenario pause banner
│           └── context/
│               ├── AuthContext.tsx   # JWT, user state, tenant switching
│               └── LanguageContext.tsx # i18n (English, Bulgarian)
├── packages/
│   └── types/                    # Shared TypeScript types
├── docker-compose.yml            # PostgreSQL container
└── package.json                  # Workspace root
```

2.3 Data Flow

1. **Usage ingestion**: Make.com scenarios call `POST /api/usage/document` with `X-API-Key` after processing documents, reporting pages spent and rows used.
2. **Daily aggregation**: Raw `DocumentUsageEvent` records are rolled up into `DocumentUsageAggregate` entries (one per tenant per day).
3. **Limit checking**: `usage-status` endpoint (polled every ~60s by the frontend) compares current-period usage against `pagesLimit + addonPagesLimit` and `rowsLimit + addonRowsLimit`.
4. **Auto-pause**: When limits are exceeded, the system calls `Make.com /scenarios/{id}/stop` for each scenario and sets `scenariosPaused = true`.
5. **Auto-resume**: When an add-on purchase or admin adjustment brings usage within limits, scenarios are automatically started via `Make.com /scenarios/{id}/start`.
6. **Monthly reset**: Lazy reset on the 5th of each month clears add-on credits and records `lastResetAt`.

---

3. Technology Stack

Layer	Technology	Version
Frontend	React	18.2
Build tool	Vite	5.0
Routing	React Router	v6
HTTP client	Axios	1.6
Charts	Recharts	2.10
Icons	Lucide React	latest
Language	TypeScript	5.3
Backend	Express.js	4.18
ORM	Prisma	5.8
Database	PostgreSQL	15
Auth	JSON Web Tokens (jsonwebtoken)	9.0
Validation	Zod	latest
Password hashing	bcryptjs	latest
Payments	Stripe	14.14
Container	Docker (PostgreSQL)	—
Package manager	npm workspaces	—

4. Database Schema

4.1 Entity Relationship Overview

```
Tenant ──1:N── Membership ──N:1── User
   │                                │
   ├── Subscription (1:1)           ├── TicketMessage
   ├── Ticket ── TicketMessage      ├── AuditLog
   │              └── Attachment    │
   ├── UsageEvent                   │
   ├── DocumentUsageEvent           │
   ├── AuditLog                     │
   └── (DocumentUsageAggregate,     │
        MonthlyUsageSnapshot,       │
        UsageAggregate — no FK)     │
```

4.2 Core Models

Tenant

The central entity representing an organisation/workspace.

Field	Type	Description
`id`	String (CUID)	Primary key
`name`	String	Organisation display name
`slug`	String (unique)	URL-safe identifier
`pagesLimit`	Int (default: 1000)	Base PDF page quota per period
`rowsLimit`	Int (default: 1000)	Base Excel row quota per period
`addonPagesLimit`	Int (default: 0)	Purchased add-on page credits
`addonRowsLimit`	Int (default: 0)	Purchased add-on row credits
`scenariosPaused`	Boolean (default: false)	Whether Make.com scenarios are paused
`lastResetAt`	DateTime?	Start of current billing period
`makeApiKey`	String?	Make.com API key
`makeOrgId`	String?	Make.com organisation ID
`makeFolderId`	String?	Make.com scenario folder filter
`azureApiKey`	String?	Azure Document Intelligence key
`azureEndpoint`	String?	Azure endpoint URL

User

Global user account (can belong to multiple tenants).

Field	Type	Description
`id`	String (CUID)	Primary key
`email`	String (unique)	Login identifier
`name`	String?	Display name
`password`	String	bcrypt-hashed password

Membership

Join table linking User to Tenant with a role.

Field	Type	Description
`userId`	String	FK → User
`tenantId`	String	FK → Tenant
`role`	Role enum	ORG_OWNER, ADMIN, BILLING_ADMIN, MEMBER, READONLY, SUPPORT_AGENT

Unique constraint: `(userId, tenantId)`

DocumentUsageEvent

Raw usage events pushed by Make.com after document processing.

Field	Type	Description
`customerId`	String	FK → Tenant
`idempotencyKey`	String	Deduplication key
`pagesSpent`	Int	PDF pages consumed
`rowsUsed`	Int	Excel rows extracted
`jobId`	String?	Make.com job identifier
`scenarioId`	String?	Make.com scenario ID
`scenarioName`	String?	Scenario display name

Unique constraint: `(customerId, idempotencyKey)`

DocumentUsageAggregate

Daily roll-up of document usage for performance.

Field	Type	Description
`customerId`	String	Tenant identifier
`date`	Date	Calendar date
`pagesSpent`	Int	Total pages that day
`rowsUsed`	Int	Total rows that day
`eventCount`	Int	Number of events aggregated

Unique constraint: `(customerId, date)`

MonthlyUsageSnapshot

Permanent monthly history (survives resets).

Field	Type	Description
`tenantId`	String	Tenant identifier
`year`	Int	Calendar year
`month`	Int	Calendar month (1-12)
`pagesSpent`	Int	Total pages that month
`rowsUsed`	Int	Total rows that month

Unique constraint: `(tenantId, year, month)`

Subscription

Field	Type	Description
`tenantId`	String (unique)	FK → Tenant
`stripeCustomerId`	String?	Stripe customer ID
`stripePriceId`	String?	Stripe price ID
`status`	String	trialing, active, canceled, past_due
`currentPeriodEnd`	DateTime?	Stripe period end

Roles (Enum)

Role	Description
`ORG_OWNER`	Full access, organisation owner
`ADMIN`	Administrative access
`BILLING_ADMIN`	Billing management only
`MEMBER`	Standard read/write access
`READONLY`	View-only access
`SUPPORT_AGENT`	Support ticket management

---

5. Authentication & Authorization

5.1 Authentication Flow

1. **Registration** (`POST /api/auth/register`): Creates User, Tenant, and Membership (role: ORG_OWNER) in a single database transaction.
2. **Login** (`POST /api/auth/login`): Verifies password with bcrypt, issues JWT containing `{ id, email, tenantId }` .
3. **Token usage**: JWT stored in `localStorage` , sent as `Authorization: Bearer <token>` on all API requests.
4. **Tenant switching** (`POST /api/auth/switch-tenant`): Issues a new JWT scoped to the selected tenant.

5.2 Authorization Model

- **Role-based access control (RBAC)** enforced by `requireRole()` middleware.
- Admin-only endpoints (user management, billing adjustments, usage reset) require `ORG_OWNER` or `ADMIN` roles.
- Tenant isolation: all database queries filter by `tenantId` extracted from the JWT.

5.3 External Authentication

Mechanism	Endpoint	Usage
`X-API-Key` header	`POST /api/usage/document`	Make.com → API usage ingestion
HMAC signature (`X-HMAC-Signature`)	`POST /v1/events/ingest`	Signed event ingestion from external systems
Stripe webhook signature	`POST /v1/billing/stripe-webhook`	Stripe → API payment notifications

---

6. API Reference

6.1 Authentication Routes (`/api/auth`)

Method	Path	Auth	Description
POST	`/register`	None	Create user + tenant
POST	`/login`	None	Authenticate, return JWT
GET	`/me`	JWT	Get user profile, tenants, integration status
POST	`/switch-tenant`	JWT	Switch active tenant, get new JWT
POST	`/profile`	JWT	Update integration keys

6.2 Billing Routes (`/v1/billing`)

Method	Path	Auth	Description
GET	`/plans`	JWT	List subscription plans
GET	`/subscription`	JWT	Current subscription status
GET	`/entitlements`	JWT	Current quotas
GET	`/usage-status`	JWT	Usage vs limits, pause state, reset date
GET	`/raw-usage`	JWT	Last 30 days raw pages & rows
POST	`/checkout`	JWT	Create Stripe checkout (placeholder)
GET	`/portal`	JWT	Stripe customer portal (placeholder)
POST	`/stripe-webhook`	Stripe sig	Handle checkout.session.completed
POST	`/reset-addon-limits`	Admin	Clear addon credits to 0
PUT	`/adjust-credits`	Admin	Add/remove usage (delta)
POST	`/simulate-addon`	Admin	Simulate add-on purchase (testing)
POST	`/reset-usage`	Admin	Delete all usage, snapshot monthly, resume scenarios

6.3 Usage Routes (`/v1/usage`)

Method	Path	Auth	Description
GET	`/summary`	Admin	Aggregated usage stats by source
GET	`/timeseries`	Admin	Daily usage for charts
GET	`/exports`	Admin	CSV export of raw events
GET	`/document-usage`	JWT	Document usage for dashboard
GET	`/openai-costs`	Admin	Real-time OpenAI cost calculation
GET	`/monthly-history`	Admin	Monthly usage snapshots

6.4 Document Usage Ingestion (`/api/usage`)

Method	Path	Auth	Description
POST	`/document`	X-API-Key	Ingest pages/rows from Make.com
GET	`/document`	X-API-Key	Query aggregated document usage

6.5 Event Ingestion (`/v1/events`)

Method	Path	Auth	Description
POST	`/ingest`	HMAC	Submit events from Make.com/Azure/OpenAI

6.6 Integration Routes (`/v1/integrations`)

Method	Path	Auth	Description
GET	`/make/check`	JWT	Verify Make.com API key
POST	`/make/sync`	JWT	Sync last 30 days usage from Make.com
POST	`/make/pause-all`	JWT	Manually pause all scenarios
POST	`/make/resume-all`	JWT	Manually resume all scenarios
GET	`/azure/check`	JWT	Verify Azure credentials

6.7 Ticket Routes (`/v1/tickets`)

Method	Path	Auth	Description
POST	`/`	JWT	Create support ticket
GET	`/`	JWT	List tickets (paginated, filterable)
GET	`/:id`	JWT	Get ticket with messages
POST	`/:id/messages`	JWT	Add message to ticket
POST	`/:id/attachments`	JWT	Get signed upload URL (placeholder)

6.8 User Routes (`/v1/users`)

Method	Path	Auth	Description
GET	`/`	Admin	List tenant members
POST	`/`	Admin	Create user in tenant
PUT	`/:membershipId/password`	Admin	Change user password
DELETE	`/:membershipId`	Admin	Remove user from tenant

---

7. Core Business Logic

7.1 Billing Period

- Period runs from the **5th of month** to the **4th of next month**.
- `getExpectedResetDate()` returns the start of the current period.
- `getNextResetDate()` returns the start of the next period.
- Monthly reset is applied **lazily** — triggered by the first `usage-status` poll after the period boundary.

7.2 Usage Limit Enforcement

```
Total limit = basePlan limit + addon credits
              (pagesLimit)    (addonPagesLimit)

If usage >= total limit:
  → Auto-pause all Make.com scenarios
  → Set scenariosPaused = true
  → Display banner in UI
```

7.3 Add-on Consumption Model (Purchased-First)

When a tenant has purchased add-on credits, usage is consumed against add-on credits **first**, then overflows to the base plan:

```
addonPagesUsed = Math.min(usage.pages, addonPages);    // Add-on fills first
basePagesUsed  = Math.max(0, usage.pages - addonPages); // Overflow to base
```

When add-on credits are fully exhausted (`addonPagesUsed >= addonPages`), the system **auto-resets** `addonPagesLimit` to 0 on the next `usage-status` poll.

7.4 Tier Mapping

Tier	Plan	Pages/month	Rows/month	Price
0	Trial / Default	5,000	5,000	Free
1	Starter	1,000	1,000	£249/mo
2	Professional	5,000	5,000	£989/mo
3	Enterprise	15,000	15,000	£2,499/mo

7.5 Auto-Pause / Auto-Resume

Auto-Pause triggers when: - `usage-status` poll detects `usage.pages >= totalPages || usage.rows >= totalRows` - Event ingestion (`/api/usage/document`) pushes usage over the limit

Auto-Resume triggers when: - Add-on purchase brings total limit above current usage - Admin adjusts credits downward - Monthly billing period resets - Admin performs a full usage reset

The system calls Make.com API `POST /scenarios/{id}/stop` or `POST /scenarios/{id}/start` for each scenario in the tenant's configured folder.

7.6 Monthly Reset Logic

On the first `usage-status` poll after the 5th of the month:

1. Compare `lastResetAt` against `getExpectedResetDate()`
2. If stale, clear `addonPagesLimit` and `addonRowsLimit` to 0
3. Update `lastResetAt` to the new period start
4. If tenant has an active subscription and scenarios were paused, auto-resume them
5. Clear `scenariosPaused` flag

---

8. Third-Party Integrations

8.1 Make.com (eu2.make.com)

Operation	API Endpoint	Method
Verify API key	`/api/v2/users/me`	GET
List organisations	`/api/v2/organizations`	GET
List scenarios	`/api/v2/scenarios?organizationId=X&folderId=Y`	GET
Get scenario usage	`/api/v2/scenarios/{id}/usage?from=&to=&interval=daily`	GET
Pause scenario	`/api/v2/scenarios/{id}/stop`	POST
Resume scenario	`/api/v2/scenarios/{id}/start`	POST

Authentication: `Authorization: Token {makeApiKey}`

8.2 Stripe

- **Webhook endpoint**: `POST /v1/billing/stripe-webhook`
- **Handled event**: `checkout.session.completed`
- **Metadata fields**: `tenantId`, `addonType` (`pages` | `rows`), `addonQuantity`
- **Fallback**: `client_reference_id` used when metadata lacks `tenantId` (for static payment links)

8.3 Azure Document Intelligence

- **Verification endpoint**: `GET {endpoint}/documentintelligence/documentModels?api-version=2024-11-30`
- **Authentication**: `Ocp-Apim-Subscription-Key` header

8.4 OpenAI

- **Usage API**: `GET https://api.openai.com/v1/usage?date={YYYY-MM-DD}`
- **Pricing**: GPT-4o at $2.50/1M input tokens, $10/1M output tokens
- **Currency conversion**: USD → EUR at fixed rate 0.93

---

9. Frontend Application

9.1 Pages

Route	Component	Description
`/`	`LandingPage`	Login / registration forms
`/dashboard`	`Dashboard`	Usage monitoring, integration settings
`/billing`	`Billing`	Subscriptions, add-ons, usage limits
`/tickets`	`Tickets`	Support ticket management
`/users`	`Users`	User management (admin only)

9.2 State Management

- **AuthContext**: JWT token, user profile, tenant list, active tenant, login/register/logout/switchTenant
- **LanguageContext**: i18n with English and Bulgarian translations, persisted to `localStorage`
- **Component-level state**: `useState` / `useCallback` / `useEffect` patterns

9.3 Real-Time Polling

`Layout.tsx` polls `GET /v1/billing/usage-status` every 60 seconds to: - Display the scenario-paused notification banner - Trigger server-side auto-pause checks - Keep usage display current

9.4 Internationalisation

Two languages supported: - **English** (`en`) — default - **Bulgarian** (`bg`)

All UI strings centralised in `LanguageContext.tsx`. Language preference persisted in `localStorage`.

---

10. Deployment & Configuration

10.1 Environment Variables

Variable	Required	Description
`DATABASE_URL`	Yes	PostgreSQL connection string
`JWT_SECRET`	Yes	Secret for JWT signing
`JWT_EXPIRES_IN`	No	Token expiry (default: `24h`)
`PORT`	No	API server port (default: `5000`)
`HMAC_SECRET`	Yes	Secret for HMAC event verification
`USAGE_API_KEY`	Yes	API key for document usage ingestion
`STRIPE_SECRET_KEY`	No	Stripe API secret key
`STRIPE_WEBHOOK_SECRET`	No	Stripe webhook signing secret
`OPENAI_API_KEY`	No	OpenAI API key for cost tracking
`MAKE_FOLDER_ID`	No	Default Make.com folder ID
`DEFAULT_TENANT_ID`	No	Fallback tenant for unauthenticated events

10.2 Development Setup

```
npm install                    # Install all workspace dependencies
docker-compose up -d           # Start PostgreSQL container
npm run db:push                # Apply Prisma schema to database
npm run dev                    # Start API (port 5000) and Web (port 5173)
```

10.3 Production Build

```
npm run build                  # TypeScript compilation (API) + Vite build (Web)
npm start                      # Start production API server
npm run preview --workspace=web # Start Vite preview server
```

10.4 Database Migrations

```
npx prisma migrate dev          # Create and apply migration (development)
npx prisma migrate deploy       # Apply pending migrations (production)
npx prisma db push              # Push schema changes without migration files
```

---

11. Error Handling & Validation

11.1 Error Response Format

```
{
  "error": {
    "message": "Human-readable error description",
    "code": "ERROR_CODE"
  }
}
```

11.2 Validation

All POST/PUT request bodies validated with **Zod** schemas. Invalid requests return 400 with descriptive error messages.

11.3 Middleware Stack

Middleware	Purpose
`authenticateToken`	Verifies JWT, attaches user to request
`requireRole(...roles)`	Checks user role against allowed roles
`verifyApiKey`	Validates `X-API-Key` header
`verifyHmacSignature`	Validates HMAC signature for event ingestion
`errorHandler`	Catches errors, formats standard response

---

12. Security Considerations

- **Password storage**: All passwords hashed with bcrypt (10 rounds).

- **JWT tokens**: Signed with HS256, configurable expiry.
- **Tenant isolation**: Every database query scoped to the authenticated tenant ID.
- **HMAC verification**: Event ingestion endpoints require cryptographic signature.
- **API key protection**: Integration keys stored per-tenant, never exposed to frontend users.
- **Stripe webhook verification**: Signature validation when `STRIPE_WEBHOOK_SECRET` is configured.
- **Input validation**: Zod schemas on all user-facing endpoints prevent injection attacks.
- **CORS**: Configured for the frontend origin only.
- **Idempotency**: Usage events deduplicated by `(tenantId, source, idempotencyKey)` composite unique constraint.

End of Technical Documentation